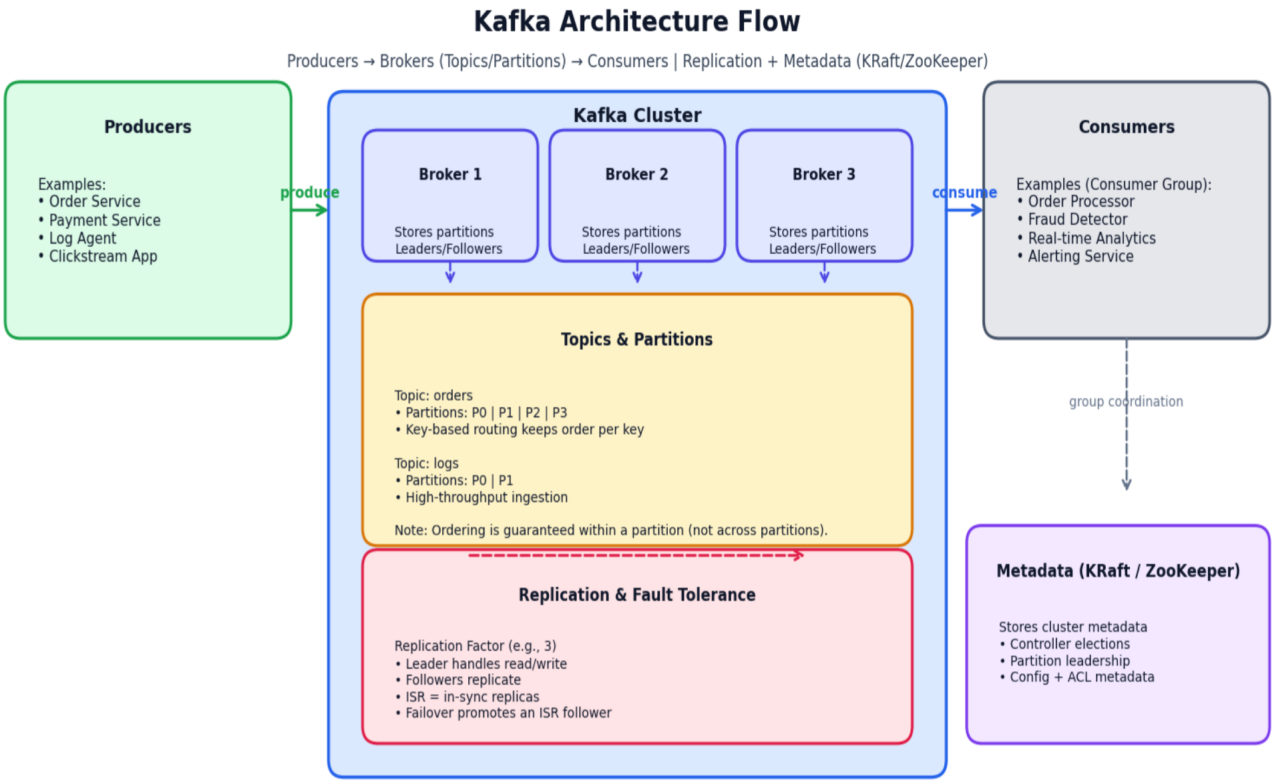
# Kafka Interview Questions & Answers

## (DevOps / Data Engineering)

*Notes for Students (Quick Revision)*

## Kafka Architecture (Core Concepts)




**Q: Explain Kafka architecture in simple terms.**

A: Kafka is a distributed streaming platform made of brokers (Kafka servers) that store data in topics. Each topic is split into partitions (ordered logs). Producers write records to partitions; consumers read records from partitions. Kafka supports replication for fault tolerance and uses ZooKeeper (older) or KRaft (newer) for cluster metadata and leader election.

**Q: What is a broker? Why do we run multiple brokers?**

A: A broker is a Kafka server that stores partitions and serves producer/consumer requests. Multiple brokers provide scalability (more storage/throughput) and fault tolerance (replicas on different brokers).

**Q: What is a topic?**

A: A topic is a named stream/category of messages (e.g., orders, payments, logs). Producers publish to a topic; consumers subscribe to a topic.

**Q: What is a partition?**

A: A partition is an ordered, append-only log within a topic. Records in a partition are strictly ordered and identified by offsets.

**Q: What are offsets?**

A: Offsets are sequential positions of records within a partition (0, 1, 2, ...). Consumers track offsets to know what has been processed and where to resume.

PATHNEX
Shaping Careers, Building Futures

**Q: What is replication and how does leader/follower work?**

A: Each partition can have multiple replicas across brokers. One replica is the leader (handles reads/writes) and others are followers (replicate the leader). If the leader fails, an in-sync follower can be promoted to leader.

**Q: What is ISR (in-sync replicas) and why is it important?**

A: ISR is the set of replicas that are fully caught up with the leader. Only ISR replicas are eligible for safe leader election and acknowledgements, improving durability and availability.

**Q: ZooKeeper vs KRaft — what's the difference?**

A: ZooKeeper was used for storing cluster metadata and controller election in older Kafka versions. KRaft replaces ZooKeeper by embedding metadata quorum and controller functionality directly into Kafka (simpler ops, fewer components).

**Q: Explain retention vs log compaction.**

A: Retention deletes old records based on time/size (e.g., keep 7 days). Log compaction keeps the latest value per key (good for state topics like user profile updates).

## Producers, Consumers, Consumer Groups

**Q: What is a producer?**

A: A producer publishes records to topics. It can choose partitions using a key (hash-based) or custom partitioner.

**Q: What is a consumer?**

A: A consumer subscribes to topics and reads records from partitions. It tracks offsets to continue processing reliably.

PATHNEX
Shaping Careers, Building Futures

**Q: What is a consumer group?**

A: A consumer group is a set of consumers that share load. Within a group, each partition is assigned to only one consumer at a time (no duplication within the group).

**Q: If a topic has 6 partitions and consumer group has 3 consumers, how will it work?**

A: Kafka assigns partitions to consumers so work is split. Example: C1 gets 2 partitions, C2 gets 2 partitions, C3 gets 2 partitions (exact mapping depends on assignor).

**Q: What if consumers > partitions?**

A: Extra consumers remain idle because a partition can't be consumed by more than one consumer in the same group at the same time.

**Q: What is rebalancing? What triggers it?**

A: Rebalancing is redistribution of partitions among consumers in a group. It happens when consumers join/leave, partitions change, or timeouts occur (e.g., consumer not polling in time).

**Q: Auto-commit vs manual commit offsets — when to use?**

A: Auto-commit is simpler but risks duplicates/loss if processing fails. Manual commit after successful processing is safer and preferred in production.

**Q: How do you guarantee message order (sequence) in Kafka?**

A: Kafka guarantees order only within a partition. For global order, use a single partition. For per-entity order (e.g., per userId/orderId), use a message key so all related messages go to the same partition, and ensure one consumer per partition in a group.

## Setup & Configuration (Single Node + Multi-node)

PATHNEX
Shaping Careers, Building Futures

**Q: How do you set up Kafka (basic steps)?**

A: Install Kafka, configure server.properties (broker.id, log.dirs, listeners, advertised.listeners, and ZooKeeper/KRaft settings), start ZooKeeper if required, start brokers, create topics, and test using console producer/consumer.

**Q: What are key configs for a multi-node cluster?**

A: Unique broker.id per broker, correct listeners and advertised.listeners, appropriate replication factor (commonly 3), sufficient disk/log.dirs, and proper monitoring/alerting.

**Q: How do clients connect to a Kafka cluster?**

A: Clients use bootstrap.servers (a list of broker endpoints). Kafka returns cluster metadata and clients then connect directly to the partition leaders.

## listeners vs advertised.listeners

**Q: What is listeners in Kafka?**

A: listeners defines where the broker binds and listens for connections (host/IP and port). Example: PLAINTEXT://0.0.0.0:9092.

**Q: What is advertised.listeners?**

A: advertised.listeners is the address/hostname the broker advertises to clients in metadata. It must be reachable by the client network.

**Q: Why is advertised.listeners important?**

A: In Docker/Kubernetes/cloud, the internal bind address may not be reachable externally. If advertised.listeners is wrong, clients fail with metadata/timeout errors because they try to connect to unreachable broker addresses.

PATHNEX
Shaping Careers, Building Futures

## Security: Authentication & Authorisation

**Q: What's the difference between authentication and authorisation?**

A: Authentication verifies identity (who you are). Authorisation controls permissions (what you can do).

**Q: How do you add authentication in Kafka?**

A: Commonly enable SASL (SCRAM/PLAIN/GSSAPI) and often TLS (SSL) for encryption. Configure broker listeners (e.g., SASL_SSL), enable mechanisms, and configure client properties accordingly.

**Q: How do you add authorisation in Kafka?**

A: Enable the authoriser on brokers and use ACLs to allow/deny operations (Read/Write/Create/Describe) on topics, groups, and clusters.

**Q: What are typical security options used in production?**

A: SASL/SCRAM (Kafka-native users), SASL/PLAIN over TLS (often with directory integration), Kerberos (SASL/GSSAPI) in enterprises, and SSL/mTLS for encryption or client cert auth.

## JAAS & Login Modules

**Q: What is JAAS in Kafka?**

A: JAAS (Java Authentication and Authorization Service) is the configuration Kafka uses to plug in authentication mechanisms (SCRAM, PLAIN, Kerberos, LDAP). Kafka loads JAAS via JVM option (e.g., KAFKA_OPTS).

**Q: What are JAAS Login Modules (sometimes called "locking modules")?**

A: They are pluggable auth components that decide how credentials are validated (e.g., ScramLoginModule, PlainLoginModule, Kerberos/GSSAPI modules, LDAP-related modules).

**Q: What do required/requisite/sufficient/optional mean in JAAS?**

A: They define control flow: required must succeed (but other modules may still run); requisite must succeed or fail immediately; sufficient can succeed and finish early (if no required failed); optional has minimal impact.

## LDAP / Active Directory (AD) Integration

**Q: How do you set up LDAP/AD authentication for Kafka?**

A: Typically use SASL/PLAIN over TLS with a JAAS configuration that validates credentials against LDAP/AD (or an organisation's standard IAM approach such as Kerberos/SSO). After authentication, enforce access using ACLs.

PATHNEX
Shaping Careers, Building Futures
Call on +91 9355165500

**Q: Which directory service (AD/LDAP) do you commonly use and why?**

A: Microsoft Active Directory is common in enterprises because identities already exist and policies are centralised. OpenLDAP may be used in smaller or Linux-heavy environments.

## Multi-Cluster / Two Kafka Clusters Communication

**Q: How do you replicate data from Kafka Cluster A to Cluster B?**

A: Use MirrorMaker 2 (recommended). It's built on Kafka Connect and replicates topics and can sync consumer group offsets for smoother failover/migration.

**Q: How do you simulate sending data from Cluster A to Cluster B?**

A: Option 1 (quick demo): consumer reads from Cluster A and produces to Cluster B. Option 2 (realistic): configure and run MirrorMaker 2 to continuously replicate from A to B, then verify by consuming from B.

PATHNEX
Shaping Careers, Building Futures

**Q: MirrorMaker 1 vs MirrorMaker 2 — what's the difference?**

A: MM1 is older and basic (consumer→producer replication, no offset sync, limited features). MM2 is Kafka Connect-based, supports offset sync, better monitoring/scaling, and more robust multi-cluster use cases (including bidirectional setups).

## Troubleshooting & Common Interview Scenarios

**Q: What is consumer lag and how do you fix it?**

A: Consumer lag is when consumption falls behind production. Fix by adding consumers (if partitions allow), increasing partitions (carefully), tuning consumer configs (fetch/max.poll), improving processing speed, using async or batching, handling retries/DLQ, and checking broker/network/disk bottlenecks.

**Q: Why do clients get 'Failed to update metadata' or timeouts?**

A: Common causes include wrong advertised.listeners, network/DNS issues, firewall rules, broker down, or TLS/SASL mismatch. Fix by ensuring brokers advertise reachable endpoints and client security configs match the broker.